

Tyr-Crypto Computational Handbook

Volume 4: BLAKE3

A guided calculation book for BLAKE3 words, G mixing, compression state, flags, chunks, tree merging, XOF output, keyed hashing, and derive-key mode.

Assumed knowledge: Volumes 0 and 1. Every new BLAKE3-specific operation is still rebuilt from small steps.

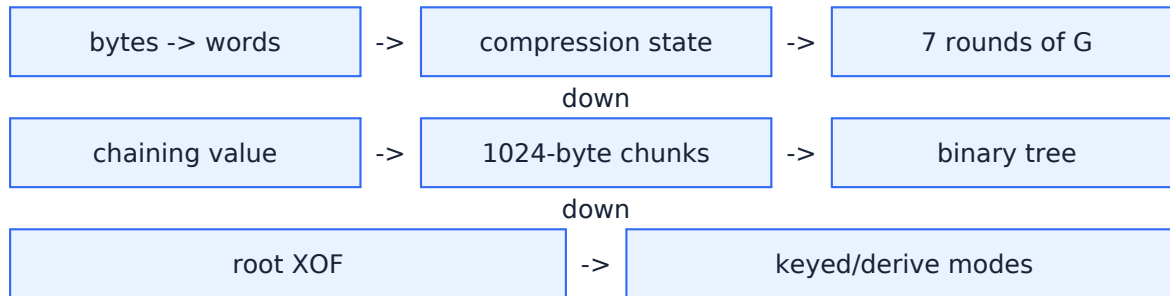
Blue	new input, public data, or plain-language explanation
Purple	exact recipe to follow
Green	fully worked calculation
Orange	exercise for the reader
Red	misuse, security, or implementation danger
Teal	checkpoint before continuing

1 What this volume will teach

Read this first

This volume does not ask you to memorize an algorithm diagram. Each page introduces exactly one mechanical action. First read the plain-language box, then copy the rule, then cover the worked examples and reproduce them yourself.

The full path



Study routine for an easily overloaded brain

1. Work for one method only. 2. Say every intermediate value aloud or write it down. 3. Stop at the teal checkpoint. 4. Continue only when the checkpoint feels boring. There is no reward for carrying an unclear step into the next page.

Vocabulary before calculations

Hash	A fixed-size or extendable fingerprint computed from input bytes.
Compression function	A fixed-input transformation that mixes a chaining value, one block, and metadata.
Chaining value (CV)	Eight 32-bit words that summarize a block, chunk, or tree node.
Block	64 input bytes, read as sixteen little-endian words.
Chunk	Up to 1024 bytes: at most sixteen blocks.
Parent node	A compression of two 32-byte child chaining values.
Flag	One public bit that labels the role or mode of a compression call.
XOF	An extendable-output function that can return more than one fixed digest length.

2 Algorithm overview before the details

The promise of the overview

The boxes below deliberately hide most arithmetic. Their job is only to show where each later calculation belongs. When you meet a calculation page, return here and point to its box.

1	bytes -> words
2	compression state
3	7 rounds of G
4	chaining value
5	1024-byte chunks
6	binary tree
7	root XOF
8	keyed/derive modes

Important scope

The examples often use toy-sized states or selected words. The toy calculation keeps the same order of operations but uses smaller numbers so that a human can finish it. Every toy rule is explicitly marked. Never substitute toy parameters into real cryptographic code.

3 Method 1: read message bytes as little-endian words

Input:	four consecutive bytes from a 64-byte BLAKE3 block
Do:	give the first byte the smallest place value
Output:	one 32-bit message word

Plain-language explanation

BLAKE3 does not mix individual printed hex bytes directly. It first groups every four bytes into one 32-bit word. “Little-endian” describes how the byte positions contribute to that word; it does not mean reversing every bit.

Mechanical rule

For bytes $b_0\ b_1\ b_2\ b_3$, compute $w = b_0 + 2^8 b_1 + 2^{16} b_2 + 2^{24} b_3$. On paper, reverse the four two-digit byte groups when writing the eight-digit word.

Worked example 1 - ordinary bytes

11 22 33 44 -> reverse byte groups -> 0x44332211.

Worked example 2 - leading zeros

09 00 00 00 -> 0x00000009. The zeros remain part of the 32-bit word.

Worked example 3 - ASCII bytes

42 4C 41 4B -> 0x4B414C42. The letters are irrelevant after they become bytes.

Worked example 4 - first key word

00 01 02 03 -> 0x03020100.

Exercises

- (a) Convert 10 20 30 40.
- (b) Convert FF 00 80 01.
- (c) Convert 54 59 52 2D.
- (d) Write the first two words of 00 01 02 03 04 05 06 07.

Checkpoint

You can point to the first byte and explain why it becomes the rightmost two hex digits of the word.

4 Method 2: perform the BLAKE3 G mixing operation

Input:	four state words a,b,c,d and two message words x,y
Do:	alternate wrapped addition, XOR, and right rotation eight times
Output:	four thoroughly mixed state words

Plain-language explanation

G is the small engine used repeatedly inside one compression round. It never creates or destroys words; it repeatedly changes four selected words. The message words x and y are injected only into the two additions that update a.

Mechanical rule

Real rule: $a = a + b + x$; $d = \text{ROTR}(d \text{ XOR } a, 16)$; $c = c + d$; $b = \text{ROTR}(b \text{ XOR } c, 12)$; $a = a + b + y$; $d = \text{ROTR}(d \text{ XOR } a, 8)$; $c = c + d$; $b = \text{ROTR}(b \text{ XOR } c, 7)$. All additions wrap modulo 2^{32} .

Worked example 1 - toy G starting 10,20,30,40

Toy rule uses 8-bit words and rotations 4,3,2,1. Add modulo 256. Steps: $a=31$; $d=17$; $c=47$; $b=EC$; $a=1F$; $d=02$; $c=49$; $b=D2$. Final $(a,b,c,d)=(1F,D2,49,02)$.

Worked example 2 - toy G starting FF,01,80,7F

Toy rule uses 8-bit words and rotations 4,3,2,1. Add modulo 256. Steps: $a=10$; $d=F6$; $c=76$; $b=EE$; $a=1E$; $d=3A$; $c=B0$; $b=2F$. Final $(a,b,c,d)=(1E,2F,B0,3A)$.

Worked example 3 - toy G starting 12,34,56,78

Toy rule uses 8-bit words and rotations 4,3,2,1. Add modulo 256. Steps: $a=E0$; $d=89$; $c=DF$; $b=7D$; $a=19$; $d=24$; $c=03$; $b=3F$. Final $(a,b,c,d)=(19,3F,03,24)$.

Worked example 4 - one real 32-bit BLAKE3 G

Use rotations 16,12,8,7 and wrap every addition at 2^{32} . Steps: $a=0x257194EC$; $d=0x61D6803E$; $c=0x9E4573B0$; $b=0xD352522D$; $a=0xF8C3E71A$; $d=0x24991567$; $c=0xC2DE8917$; $b=0x742319B6$. Final $(0xF8C3E71A, 0x742319B6, 0xC2DE8917, 0x24991567)$.

Exercises

- (a) Toy G8: $a=00, b=00, c=00, d=00, x=01, y=00$.
- (b) Toy G8: $a=01, b=02, c=03, d=04, x=05, y=06$.
- (c) Toy G8: $a=AA, b=55, c=0F, d=F0, x=11, y=22$.
- (d) In the real G rule, which two updates directly add message words?

Checkpoint

You can execute exactly one line at a time and never reuse an old value after a line has overwritten it.

5 Method 3: build the 16-word compression state

Input:	8 chaining-value words, counter, block length, and flags
Do:	place each input into its fixed state position
Output:	the starting state $v[0..15]$

Plain-language explanation

The compression function needs more than the message block. The lower half remembers where this block belongs: which chunk, how many bytes are valid, and whether this is a start, end, parent, or root operation.

Mechanical rule

Set $v[0..7]=CV[0..7]$. Set $v[8..11]=IV[0..3]$. Set $v[12]$ =low 32 bits of counter, $v[13]$ =high 32 bits, $v[14]$ =block length, $v[15]$ =flags.

Worked example 1 - first full block

counter=0, len=64, CHUNK_START=1 -> $v_{12}=0$, $v_{13}=0$, $v_{14}=0x40$, $v_{15}=0x01$.

Worked example 2 - last short block

counter=3, len=17, CHUNK_END=2 -> $v_{12}=3$, $v_{13}=0$, $v_{14}=0x11$, $v_{15}=0x02$.

Worked example 3 - large counter

counter=0x00000002_00000005 -> $v_{12}=0x00000005$ and $v_{13}=0x00000002$.

Worked example 4 - parent node

counter=0, len=64, PARENT=4 -> $v_{12}=0$, $v_{13}=0$, $v_{14}=0x40$, $v_{15}=0x04$.

Exercises

- (a) Place counter 7, len 64, flags 3 into $v_{12}..v_{15}$.
- (b) Split counter 0x00000001_00000000.
- (c) Place counter 9, len 0, flags 3.
- (d) Which IV words are copied into the state: all eight or only four?

Checkpoint

You can reconstruct $v_{12}, v_{13}, v_{14}, v_{15}$ without looking at code.

6 Method 4: permute the message schedule between rounds

Input:	the current list of sixteen message-word indices
Do:	read old positions in the fixed order P
Output:	the schedule for the next round

Plain-language explanation

A compression round uses all sixteen message words, but the next round connects them to different G calls. BLAKE3 obtains each new order by applying one fixed permutation to the previous schedule.

Mechanical rule

$P=[2,6,3,10,7,0,4,13,1,11,12,5,9,14,15,8]$. If old schedule is s , then $\text{new}[i]=s[P[i]]$. Do not replace indices one at a time in the same array; build a fresh list.

Worked example 1 - round 0 to round 1

$[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15] \rightarrow [2, 6, 3, 10, 7, 0, 4, 13, 1, 11, 12, 5, 9, 14, 15, 8]$.

Worked example 2 - round 1 to round 2

$[2, 6, 3, 10, 7, 0, 4, 13, 1, 11, 12, 5, 9, 14, 15, 8] \rightarrow [3, 4, 10, 12, 13, 2, 7, 14, 6, 5, 9, 0, 11, 15, 8, 1]$.

Worked example 3 - round 2 to round 3

$[3, 4, 10, 12, 13, 2, 7, 14, 6, 5, 9, 0, 11, 15, 8, 1] \rightarrow [10, 7, 12, 9, 14, 3, 13, 15, 4, 0, 11, 2, 5, 8, 1, 6]$.

Worked example 4 - track one word

In the first permutation, old word m_0 appears where $P[i]=0$. This occurs at $i=5$, so m_0 moves to schedule position 5.

Exercises

- What occupies new position 0 from the initial schedule?
- Where does m_8 move after the first permutation?
- Compute the first four entries after two permutations.
- Why should a fresh schedule array be used?

Checkpoint

You can compute a new schedule by indexing the old schedule, not by guessing a shuffle pattern.

7 Method 5: wire one round into column and diagonal G calls

Input:	the 4x4 state positions and the current message schedule
Do:	select four columns, then four diagonals
Output:	eight G calls that touch every state word twice

Plain-language explanation

The round is not sixteen independent calculations. First four G calls run down the columns of the 4x4 state. The next four run along diagonals. This changes which words can influence one another.

Mechanical rule

Columns: G(0,4,8,12;m0,m1), G(1,5,9,13;m2,m3), G(2,6,10,14;m4,m5), G(3,7,11,15;m6,m7). Diagonals: G(0,5,10,15;m8,m9), G(1,6,11,12;m10,m11), G(2,7,8,13;m12,m13), G(3,4,9,14;m14,m15). Replace m positions by the current schedule.

Worked example 1 - first column

State cells 0,4,8,12 receive schedule words at positions 0 and 1.

Worked example 2 - fourth column

State cells 3,7,11,15 receive schedule positions 6 and 7.

Worked example 3 - first diagonal

State cells 0,5,10,15 receive schedule positions 8 and 9.

Worked example 4 - round-1 first column

Round-1 schedule begins [2, 6], so the first G injects m2 and m6.

Exercises

- List state cells of the second column G.
- List state cells of the third diagonal G.
- Using round-1 schedule, which words enter the second column?
- How many times is state word v0 touched in one round?

Checkpoint

Given a state index, you can find both G calls that update it during one round.

8 Method 6: turn the final state into output and a chaining value

Input:	final 16 state words plus the input CV
Do:	XOR the state halves and then XOR the upper half with the input CV
Output:	16 output words; the first 8 are the next CV

Plain-language explanation

The compression state is not returned unchanged. The final XOR step folds the two halves together. Hash-tree calculations keep only the first eight output words as a 256-bit chaining value.

Mechanical rule

For $i=0..7$: $\text{out}[i]=\text{state}[i] \text{ XOR } \text{state}[i+8]$. Also $\text{out}[i+8]=\text{state}[i+8] \text{ XOR } \text{inputCV}[i]$. The chaining value is $\text{out}[0..7]$.

Worked example 1 - one folded word

$\text{state0}=0x11223344$, $\text{state8}=0xFFFF0000 \rightarrow \text{out0}=0xEEDD3344$.

Worked example 2 - upper output

$\text{state8}=0x0F0F0F0F$, $\text{cv0}=0x33333333 \rightarrow \text{out8}=0x3C3C3C3C$.

Worked example 3 - all-zero lower word

$\text{state2}=0$, $\text{state10}=0x12345678 \rightarrow \text{out2}=0x12345678$.

Worked example 4 - equal words cancel

$\text{state5}=\text{state13}=0xA5A5A5A5 \rightarrow \text{out5}=0$.

Exercises

- (a) $\text{state1}=0xAAAAAAAA$, $\text{state9}=0x55555555$: out1?
- (b) $\text{state12}=0x12340000$, $\text{cv4}=0x00FF00FF$: out12?
- (c) If $\text{state3}=\text{state11}$, what is CV word 3?
- (d) Which output words form the chaining value?

Checkpoint

You can say why BLAKE3 stores only eight words for a tree node although compression produces sixteen.

9 Method 7: combine BLAKE3 flags

Input:	one or more named conditions
Do:	OR their power-of-two flag values
Output:	one integer in state word v15

Plain-language explanation

Flags are switches packed into one number. Because every flag uses a different bit, OR combines them without ambiguity. A flag sum is not secret arithmetic; it records the role of the current compression call.

Mechanical rule

CHUNK_START=1, CHUNK_END=2, PARENT=4, ROOT=8, KEYED_HASH=16, DERIVE_KEY_CONTEXT=32, DERIVE_KEY_MATERIAL=64. Combine with bitwise OR.

Worked example 1 - single start

CHUNK_START -> 1.

Worked example 2 - single-block chunk

START OR END = 1 OR 2 -> 3.

Worked example 3 - root parent

PARENT OR ROOT = 4 OR 8 -> 12.

Worked example 4 - keyed single-block root

START OR END OR ROOT OR KEYED = 1+2+8+16 -> 27.

Exercises

- (a) END OR ROOT.
- (b) PARENT OR KEYED_HASH.
- (c) START OR END OR DERIVE_KEY_CONTEXT.
- (d) Decode flag value 72.

Checkpoint

You can decode any flag integer by subtracting the largest present power of two.

10 Method 8: split a message into blocks and chunks

Input:	message length in bytes
Do:	group 64-byte blocks into 1024-byte chunks and mark boundaries
Output:	block counts, chunk counters, and start/end flags

Plain-language explanation

A chunk holds at most sixteen 64-byte blocks. Each chunk gets its own counter. The first block of a chunk receives `CHUNK_START`; the final block receives `CHUNK_END`. A one-block chunk receives both.

Mechanical rule

$\text{chunks} = \max(1, \text{ceil}(\text{length}/1024))$. Within each chunk, $\text{blocks} = \max(1, \text{ceil}(\text{chunkBytes}/64))$. Counter is the zero-based chunk number. Last block length is the remaining byte count, or 64 when exactly full.

Worked example 1 - empty input

One chunk, one zero-length block, counter 0, flags `START|END=3`.

Worked example 2 - 20 bytes

One chunk, one block of length 20, flags 3.

Worked example 3 - 130 bytes

One chunk with block lengths 64,64,2. Flags: 1,0,2.

Worked example 4 - 1100 bytes

Chunk 0 has 1024 bytes and 16 blocks. Chunk 1 has 76 bytes and two blocks with lengths 64 and 12.

Exercises

- (a) Length 64: list blocks and flags.
- (b) Length 65: list block lengths and flags.
- (c) Length 2048: number of chunks and blocks.
- (d) Length 2050: final chunk counter and final block length.

Checkpoint

You can decide `START` and `END` from position inside a chunk, not from the message as a whole.

11 Method 9: merge chunk chaining values into a binary tree

Input:	a sequence of completed chunk CVs
Do:	merge equal-height neighbors with PARENT compression
Output:	one root node plus possibly an unfinished stack during streaming

Plain-language explanation

BLAKE3 keeps a stack of subtrees. A new chunk starts at height 0. Whenever the two top stack entries have equal heights, they are merged into one parent one level higher. This is binary carrying: two 1s become one 2.

Mechanical rule

Push (CV,level=0). While top two levels are equal: remove right and left, compute Parent(left,right), and push the result at level+1. Preserve left/right order.

Worked example 1 - one chunk

Push C0@0. Stack: [C0@0]. No merge.

Worked example 2 - two chunks

Push C1@0 beside C0@0 -> merge to P01@1. Stack [P01@1].

Worked example 3 - three chunks

After P01@1, push C2@0. Levels differ, so stack [P01@1,C2@0].

Worked example 4 - four chunks

Push C3@0 -> C2+C3=P23@1; then P01+P23=Root@2.

Exercises

- Show stack levels after 5 chunks.
- Show stack levels after 6 chunks.
- Which node is left when merging chunks 2 and 3?
- Why may the rightmost incomplete subtree remain unmerged during streaming?

Checkpoint

You can reproduce tree merges using only the sequence of stack levels.

12 Method 10: produce extendable root output

Input:	the final root Output node and requested byte length
Do:	compress repeatedly with ROOT flag and output-block counter 0,1,2,...
Output:	as many output bytes as requested

Plain-language explanation

BLAKE3 is an XOF: the digest does not have to stop at 32 bytes. Each root compression emits 64 bytes. Longer output changes only the root output counter, not the message tree.

Mechanical rule

Set flags=output.flags OR ROOT. For blockCounter=0,1,2,... compress the same root node and serialize 16 words little-endian. Copy min(64,bytes remaining).

Worked example 1 - 32-byte digest

Use output counter 0 and copy its first 32 bytes.

Worked example 2 - 64-byte output

Use counter 0 and copy all 64 bytes.

Worked example 3 - 80-byte output

Use counter 0 for 64 bytes, then counter 1 for 16 bytes.

Worked example 4 - 130-byte output

Counters 0 and 1 give 128 bytes; counter 2 supplies the final 2 bytes.

Exercises

- (a) How many root blocks for 1 byte?
- (b) How many for 65 bytes?
- (c) Counters used for 192 bytes?
- (d) Does requesting 100 bytes change chunk processing?

Checkpoint

You can calculate output-block count as $\text{ceil}(\text{outLen}/64)$.

13 Method 11: select plain, keyed, and derive-key modes

Input:	purpose, optional 32-byte key, and input bytes
Do:	choose initial CV and mode flags; derive-key uses two phases
Output:	hash, keyed output, or domain-specific derived material

Plain-language explanation

The compression arithmetic stays the same. Modes change the initial chaining value and a flag. Derive-key intentionally hashes a textual context first and uses that 32-byte result as the key for a second hash of the material.

Mechanical rule

Plain: key=IV, flags 0. Keyed: key=the eight little-endian key words, flags 16. Derive context: key=IV, flags 32, output 32 bytes. Derive material: use context output as key, flags 64.

Worked example 1 - plain hash

Input "abc"; IV as key words; no mode flag.

Worked example 2 - keyed hash

Key bytes 00..1F become words 0x03020100, 0x07060504, 0x0B0A0908, 0x0F0E0D0C, 0x13121110, 0x17161514, 0x1B1A1918, 0x1F1E1D1C. Use KEYED_HASH=16.

Worked example 3 - derive context

Hash context string with DERIVE_KEY_CONTEXT=32 to obtain exactly 32 context-key bytes.

Worked example 4 - derive material

Interpret context key as eight little-endian words, hash material with DERIVE_KEY_MATERIAL=64.

Exercises

- Which mode accepts no key and uses flag 0?
- What key length does keyed mode require?
- List the two derive-key phases.
- May the derive-context phase accept a caller key in Tyr-Crypto?

Checkpoint

You can explain why derive-key is not simply "keyed hash with the context string as the key."

14 Cumulative guided calculations

How cumulative work differs

Earlier exercises isolated one action. These tasks chain several actions together. Draw a small checkbox after every line. Do not calculate two lines in your head at once.

Cumulative task 1 - One 70-byte chunk

Split a 70-byte message into blocks. Give each block length and flags. Then state which output words become the chunk CV.

Cumulative task 2 - Five-chunk tree

Write the stack-level sequence after each of five chunk CVs arrives.

Cumulative task 3 - Keyed 80-byte XOF

State the base flag, root flag combination, number of output blocks, and output counters.

Cumulative task 4 - Derive-key pipeline

Describe every key and flag used for context C and material M.

15 Tyr-Crypto code map

How to read the code map

The left column names the calculation from this volume. The right column names the current Nim location or implementation behavior. This is a map, not a claim that the implementation is independently audited.

Little-endian load/store	src/protocols/custom_crypto/symmetric/blake3/blake3.nim: load32, store32
G mixing and message permutation	blake3.nim: g, permute, compress
Chunk construction	blake3.nim: chunkOutput; blockLen=64, chunkLen=1024
Tree stack	blake3.nim: pushCv, parentOutput, processChunkBytes
Root XOF	blake3.nim: outputBytes, finalBlake3
Modes	blake3.nim: Blake3Mode, blake3Digest, blake3DeriveKey
SIMD batching	blake3_simd.nim and blake3CompressBatch

Security and implementation note

Tyr-Crypto selects scalar, SSE2, NEON, or AVX2 compression backends. SIMD changes how several independent lanes are executed; it must not change any mathematical result.

Security and implementation note

The current hasher stores a 1024-byte chunk buffer and stacks of chaining values. Short-lived context-key material in derive-key mode is explicitly cleared, but language/runtime copies and compiler behavior remain part of the broader key-lifetime problem.

16 Compact solutions with paths

Use solutions correctly

First compare the final answer. If it differs, compare one intermediate value at a time from left to right. Do not copy the whole line: locate the first point where your work diverged.

Method 1: read message bytes as little-endian words

Solution path

- (a) 0x40302010.
- (b) 0x018000FF.
- (c) 0x2D525954.
- (d) $w_0=0x03020100$ and $w_1=0x07060504$.

Method 2: perform the BLAKE3 G mixing operation

Solution path

- (a) Final (03,6B,D4,C4).
- (b) Final (46,2E,64,A1).
- (c) Final (3B,B1,6A,4D).
- (d) The first and fifth lines: $a=a+b+x$ and $a=a+b+y$.

Method 3: build the 16-word compression state

Solution path

- (a) $v_{12}=7$, $v_{13}=0$, $v_{14}=64$, $v_{15}=3$.
- (b) $v_{12}=0$, $v_{13}=1$.
- (c) $v_{12}=9$, $v_{13}=0$, $v_{14}=0$, $v_{15}=3$.
- (d) Only $IV[0..3]$ go to $v[8..11]$.

Method 4: permute the message schedule between rounds

Solution path

- (a) m_2 , because $P[0]=2$.
- (b) m_8 moves to position 15.
- (c) $[3, 4, 10, 12]$.
- (d) In-place overwriting would destroy old entries that later positions still need.

Method 5: wire one round into column and diagonal G calls

Solution path

- (a) (1,5,9,13).
- (b) (2,7,8,13).
- (c) m3 and m10.
- (d) Twice: first column and first diagonal.

Method 6: turn the final state into output and a chaining value

Solution path

- (a) 0xFFFFFFFF.
- (b) 0x12CB00FF.
- (c) 0.
- (d) out[0..7].

Method 7: combine BLAKE3 flags

Solution path

- (a) $2 \text{ OR } 8 = 10$.
- (b) $4 \text{ OR } 16 = 20$.
- (c) $1 \text{ OR } 2 \text{ OR } 32 = 35$.
- (d) $72=64+8$, so `DERIVE_KEY_MATERIAL` and `ROOT`.

Method 8: split a message into blocks and chunks

Solution path

- (a) One 64-byte block, flags 3.
- (b) Lengths 64 and 1; flags 1 then 2.
- (c) 2 chunks, 32 blocks total.
- (d) Final chunk counter 2; its only block has length 2 and flags 3.

Method 9: merge chunk chaining values into a binary tree

Solution path

- (a) After 4 gives [2]; push fifth -> [2,0].
- (b) After fifth [2,0]; sixth merges two level-0 -> [2,1].
- (c) C2 is left; C3 is right.
- (d) It has no equal-height neighbor yet; future chunks may complete it.

Method 10: produce extendable root output

Solution path

- (a) 1 block.
- (b) 2 blocks.
- (c) Counters 0,1,2.
- (d) No. The same root node is expanded for longer output.

Method 11: select plain, keyed, and derive-key modes

Solution path

- (a) Plain hash.
- (b) 32 bytes.
- (c) Context hash with flag 32; material hash keyed by the context result with flag 64.
- (d) No; the context phase uses the IV and rejects a supplied key.

17 Cumulative solutions

Cumulative solution 1 - One 70-byte chunk

Blocks: 64 bytes with `CHUNK_START=1`, then 6 bytes with `CHUNK_END=2`. Both use chunk counter 0. After the last compression, output words 0..7 form the chunk CV.

Cumulative solution 2 - Five-chunk tree

After C0: [0]. C1: [1]. C2: [1,0]. C3: [2]. C4: [2,0].

Cumulative solution 3 - Keyed 80-byte XOF

Base mode flag=16. Root expansion uses 16 OR 8 plus the root node flags. 80 bytes need two 64-byte output blocks with counters 0 and 1; copy 64 then 16 bytes.

Cumulative solution 4 - Derive-key pipeline

Phase 1: hash C from IV with flag 32, output 32 bytes. Phase 2: parse those bytes as eight little-endian key words and hash M with flag 64. Add ordinary chunk/parent/root flags as required by the tree and output stage.

18 Primary references

Tyr-Crypto BLAKE3 implementation	https://github.com/siriuslee69/Tyr-Crypto/blob/main/src/protocols/custom_crypto/symmetric/blake3/blake3.nim
BLAKE3 specification repository	https://github.com/BLAKE3-team/BLAKE3-specs
BLAKE3 official implementation	https://github.com/BLAKE3-team/BLAKE3
Tyr-Crypto algorithm inventory	https://github.com/siriuslee69/Tyr-Crypto/blob/main/docs/ALGORITHMS.md

Safety boundary

This handbook explains calculations and implementation structure. It does not certify Tyr-Crypto or any custom cryptographic implementation for production use. Protocol composition, randomness, nonce management, compiler behavior, side channels, key erasure, and independent review remain separate requirements.